

Using Logistic Regression and XGBoost for Bot Detection

Elias Schlie

u783766

Tilburg University

e.p.schlie@tilburguniversity.edu

Abstract

The activity of bots on social media platforms is increasingly becoming a problem and platform providers are seeking ways to prevent bots from infiltrating their systems. The following project evaluates two simple machine learning algorithms on their ability to discern between bots and users based on their usage data. The algorithms applied are logistic regression, which was able to reach an accuracy of 97%, and XGBoost, which reached 98% accuracy.

1 Introduction

With the rise of large language models, bots now have the ability to immitate human behavior better than ever before. This makes discerning them from actual users increasingly hard, and more sophisticated methods are needed to detect them.

2 Data and Preprocessing

2.1 Dataset Description

The "bots_vs_users.csv" dataset contains information on 5874 social media accounts, exactly half of which are bots. Every account has 59 features, that include 14 numerical ones like their `average_views` and 45 categorical ones like the city they live in. 41 of the categorical features are binary like the the variable `has_profile_picture` for example. Most of the features include a large number of missing or unknown values. 4483 accounts don't contain any numerical information, while for the individual categorical features, between 5006 and 24 of the accounts carry the label "unknown". Only the target variable "target", which indicates if the account is a bot or not has no missing features.

2.2 Data split

Before any preprocessing was applied, the data was split into a training and a test set using a stratified 80/20 split. This was done prior to preprocessing due to the reason that some of the preprocess-

ing steps learn from the data and thus would lead to data leakage if applied to the entire dataset at once. Model hyperparameters were validated based on a k-fold cross validation on the training set. To ensure reproducibility, this and every following step that involves randomness was seeded with the seed 42.

2.3 Preprocessing and Feature Engineering

All the preprocessing and feature engineering steps were implemented in a single `DataCleaner` class. The class's `fit_transform` method was used on the training data and transforms it while learning distributional features from it for functions like scaling. The `transform` method applies the same transformations with the previously learned features and was used on the testing dataset. This sections lists all the transformations in the order they were applied.

Minimal profile flag A binary categorical feature was added, discerning between the accounts that have numeric features and the accounts without them. This step was done first because later steps add further numeric variables, which have missing values for different users, but the intended variable was intended to capture exactly the set of users that don't have numeric features in the initial dataset.

Subscribers Count Feature While reflecting a numeric quantity, the `subscribers_count` feature was stored as a categorical variable with 1597 possi-

ble categories. To enable the machine learning algorithms, to learn from the numeric relationships, this variable was converted to be numeric.

Data Scaling To ensure equal contribution of features and faster convergence for the logistic regressor, the data was scaled with the `StandardScaler` of the `sklearn` python library.

Median imputation The `SimpleImputer` class of the `sklearn` library was used to replace all missing values for numerical features with the median of this feature. That way, it is possible for the logistic regressor to use the numeric features without creating unnecessary noise.

Categorical encoding Categorical features with exactly two categories were binary encoded, mapping each category to 0 or 1. The remaining categories were one-hot encoded, while assigning all categories with less than 30 members into an `other` bin. This `other` bin had the effect of reducing the number of city categories from 362 to only 6 and thereby preventing a drastic dimensionality increase that would have otherwise come from one-hot encoding it. For all categorical features, the `unknown` category was treated just as any other category.

3 Models

3.1 Baseline

For a baseline measurement, `sklearn`'s `DummyClassifier` was used with the "stratified" strategy. This strategy randomly predicts labels based on the frequency with which they occur in the dataset. It achieved an accuracy of 0.5 and an F1-score of 0.49 on the bot class.

3.2 Logistic Regressor

Logistic regression is one of the simplest machine learning algorithms. Despite its simplicity, it performs surprisingly well on linearly separable classification tasks. A major advantage of logistic regression is that its coefficients directly reflect feature importance for the classification task and can thus easily be interpreted. Additionally, it is very computationally efficient when dealing with large feature spaces. This makes it a good choice for local hyperparameter tuning and evaluation on the current dataset.

3.3 XGBoost

XGBoost (Extreme Gradient Boosting) is a high-performance implementation of gradient-boosted decision trees designed for speed and efficiency. (Chen & Guestrin, 2016) As other decision tree derived algorithms, it is particularly well-suited for structured datasets with a mix of categorical and numerical features, as in the current case.

3.4 Model Comparison

The most striking difference between logistic regression and XGBoost is their ability to handle non-linear data. While XGBoost can pick up on complex non-linear relationships, logistic regression doesn't have this ability. Next to this difference, the two algorithms are similar in that they both are very compute efficient and use regularization to decrease overfitting, which is essential when working with high dimensional data.

4 Model Training

4.1 Logistic Regression

A critical hyperparameter in logistic regression is `C`, which controls the strength of regularization. Lower `C` values impose stronger regularization, shrinking the model's coefficients toward zero (in the case of the default L2 penalty), and thereby reducing the risk of overfitting.

A grid search was performed using `GridSearchCV` from `scikit-learn` to select the optimal regularization parameter `C`, testing the values 0.01, 0.1, 0.25, 0.5, 1, 2.5, 5, 10, 20, 40, and 80. `C = 2.5` was identified as the best value and a model was trained on the whole training set.

As visible in Figure A.1 in the appendix, the average F1 score across five folds peaks around the value of 2.5, confirming the value arrived at with grid-search. Nevertheless, the error bars clearly indicate that the effect is extremely small. The difference between the worst and the best mean F1 score is only around 0.004%, indicating, that regularization strength doesn't have a strong impact on performance in this dataset.

The final linear regressor was trained on the entire training set with a `C` value of 2.5.

4.2 XGBoost

With way more relevant hyperparameters than logistic regression, leading to a combinatorial explosion,

gridsearch proved unsuitable and Optuna (Akiba et al., 2019) was used instead. Optuna is an open-source hyperparameter optimizing framework that can efficiently search through a large space of possible hyperparameter combinations. It uses advanced strategies like Bayesian optimization to intelligently explore the search space, thereby significantly reducing the number of tests that need to be done to arrive at a good result.

To ensure reliable results, every hyperparameter combination was scored on its average F1 score over 5 independently seeded 3-fold cross-validations. Optuna conducted a total of 100 trials with different combinations of hyperparameters. The best performing configuration achieved an average F1 score of 0.9781. Search ranges for hyperparameters and the resulting best combination can be found in Table 1.

Table 1: Best hyperparameter configuration for XGBoost and corresponding search ranges

Hyperparameter	Best Value	Search Range
n_estimators	281	[50, 500]
learning_rate	0.2003	[0.01, 0.3]
max_depth	3	[3, 8]
min_child_weight	1	[1, 5]
subsample	0.9284	[0.6, 1.0]
colsample_bytree	0.6527	[0.6, 1.0]
gamma	1.0618	[0, 5]

Like previously with the logistic regressor, variance between the best and worst set of hyperparameters was only minimal, this time spanning a little more than 0.005% in average F1 performance. Due to the fact that different hyperparameters are interacting, looking at them separately from each other becomes less meaningful and harder to interpret. Figure A.2 attempts this by visualizing how learning rate and gamma influence F1 score. It reveals, that a learning rate around 0.2 results in slightly higher F1 scores while higher gamma values tend to slightly decrease prediction performance.

The final model used the best parameters listed in Table 1 column two and was re-trained on the entire training set.

5 Results

5.1 Logistic regression

On the hold-out test set, the logistic regressor reached an F1 score of 0.9724, accuracy of 0.9728, precision of 0.9843, recall of 0.9608, and an area under

curve (AUC) for the receiver operating characteristic (ROC) of 0.9951. Table A.1 displays the confusion matrix of the predictions, revealing that only 23 out of 587 bots got misclassified as users. A ROC curve and a precision-recall curve can be found in figure A.4 and figure A.3 of the appendix respectively.

5.2 XGBoost

XGBoost was able to achieve an F1 score of 0.9768, accuracy of 0.9770, precision of 0.9861, recall of 0.9676, and ROC-AUC of 0.9962. Those metrics are reflected in its confusion matrix (Table A.2) that shows only 19 misclassified bots. Like previously with the logistic regression, the ROC curve (figure A.6) and a precision-recall curve (figure A.5) are very close to the optimum.

6 Discussion

6.1 Model Performance Comparison

While XGBoost performs slightly better on all metrics, this difference is extremely small. With only 19 instead of 23 misclassified bots and only 8 instead of 9 misclassified users, XGBoosts accuracy is 0.0042 points higher than the one of logistic regression. This is also reflected in F1 score, where the difference increases slightly to 0.0044 points. Those differences are incredibly small and clearly show that XGBoosts ability to find non-linear relationships in data is not majorly critical for the given dataset.

6.2 Generalizability

Both models seem to generalize extremely well and don't show signs of overfitting to the training data. Comparing the F1 scores during hyperparameter tuning and testing shows only minimal differences of 0.0013 points in the F1 score of XGBoost. This difference is extremely small, especially when considering that hyperparameters were specifically chosen to fit the training set as well as possible.

6.3 Feature Analysis

7 Conclusion

Test

References

- Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2623–2631. <https://doi.org/10.1145/3292500.3330701>
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>

A Tables

Table A.1: Confusion matrix for logistic regression predictions on the test set

	Predicted Users	Predicted Bots
Users	579	9
Bots	23	564

Table A.2: Confusion matrix for XGBoost predictions on the test set

	Predicted Users	Predicted Bots
Users	580	8
Bots	19	568

Table A.3: 10 most important features for XGBoost predictions based on their

	0
has_photo_0.0	741.480
has_mobile_Unknown	366.661
has_status_Unknown	354.188
all_posts_visible_Unknown	232.566
subscribers_count	37.060
avg_likes	33.168
city_Saint Petersburg	27.401
has_website_0.0	26.403
has_occupation	18.723
has_status_0.0	16.661

B Figures

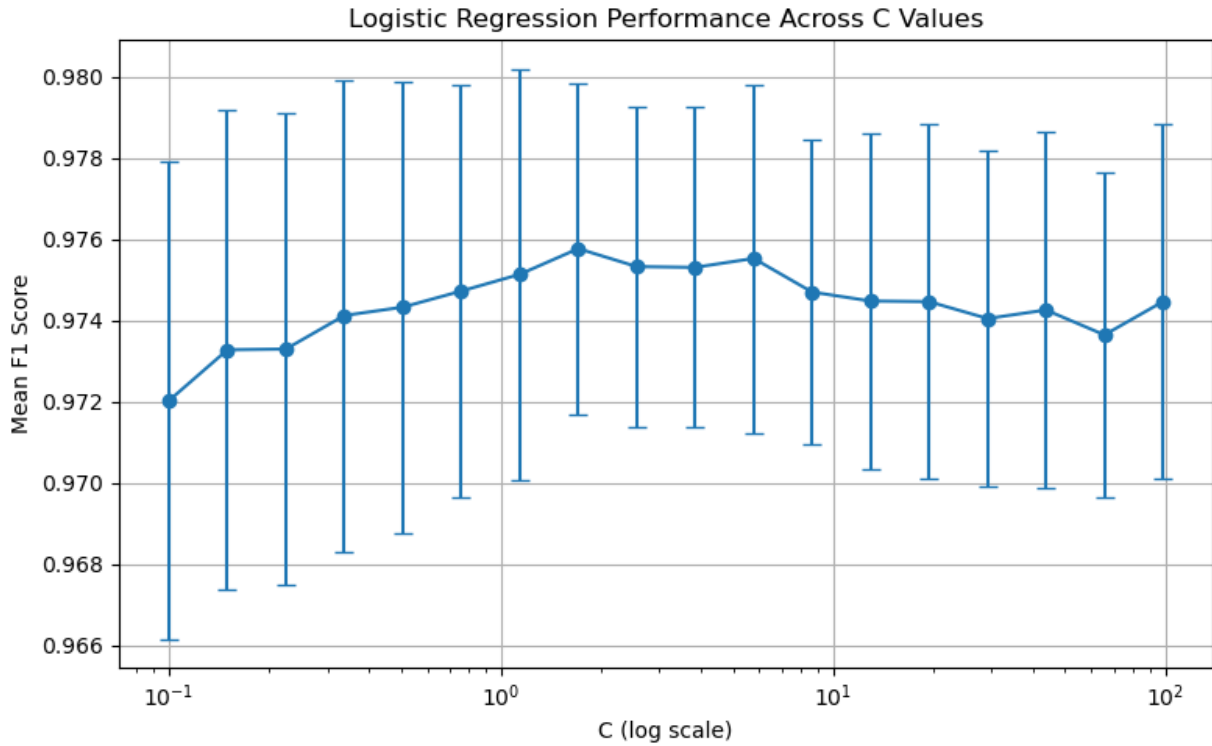


Figure A.1: Mean F1 score of logistic regression across different regularization strengths (C), evaluated using 5-fold stratified cross-validation. Error bars represent the standard deviation across folds. A log scale is used on the x-axis to capture the exponential range of tested C values. While a peak around 2.5 is visible, the error bars suggest that the effect of C is only minimal.

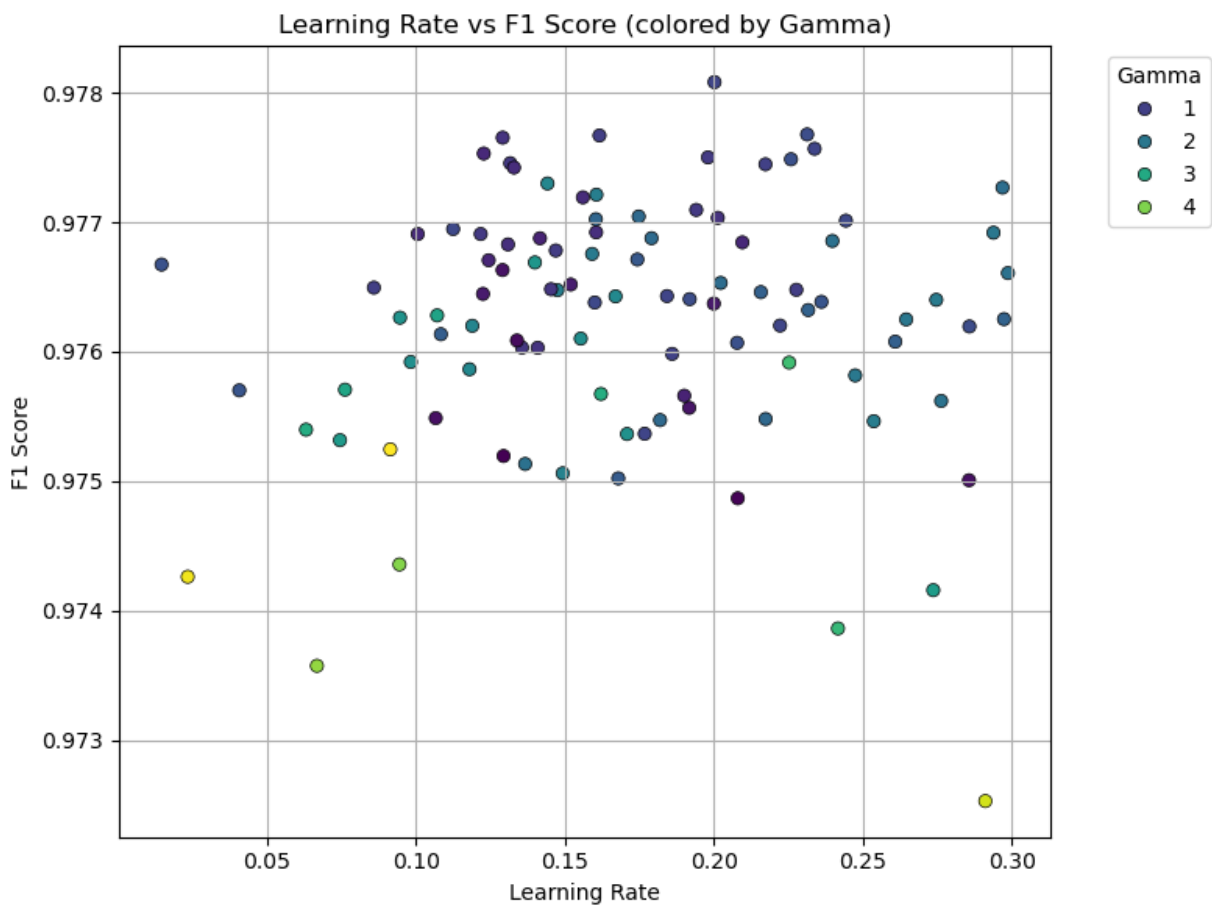


Figure A.2: Scatterplot of the effect of the XGBoost hyperparameters `learning_rate` and `gamma` on average F1 score.

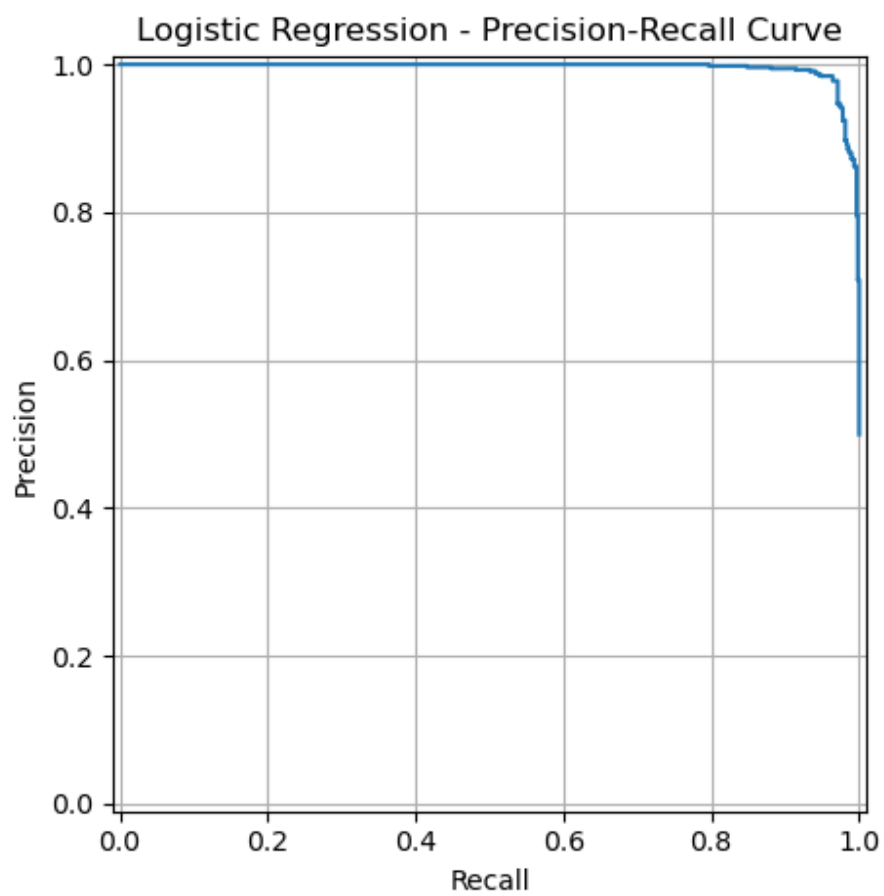


Figure A.3: Precision recall curve of the logistic regressor on the hold-out test set.

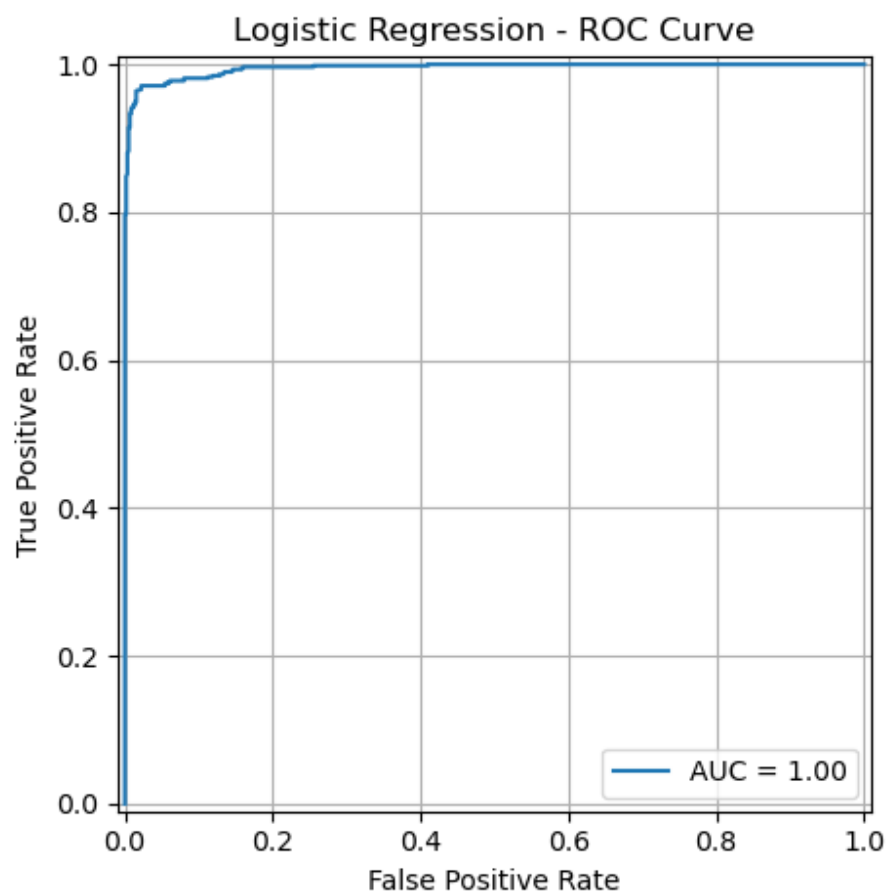


Figure A.4: Receiver operating characteristic curve of the logistic regressor on the hold-out test set.

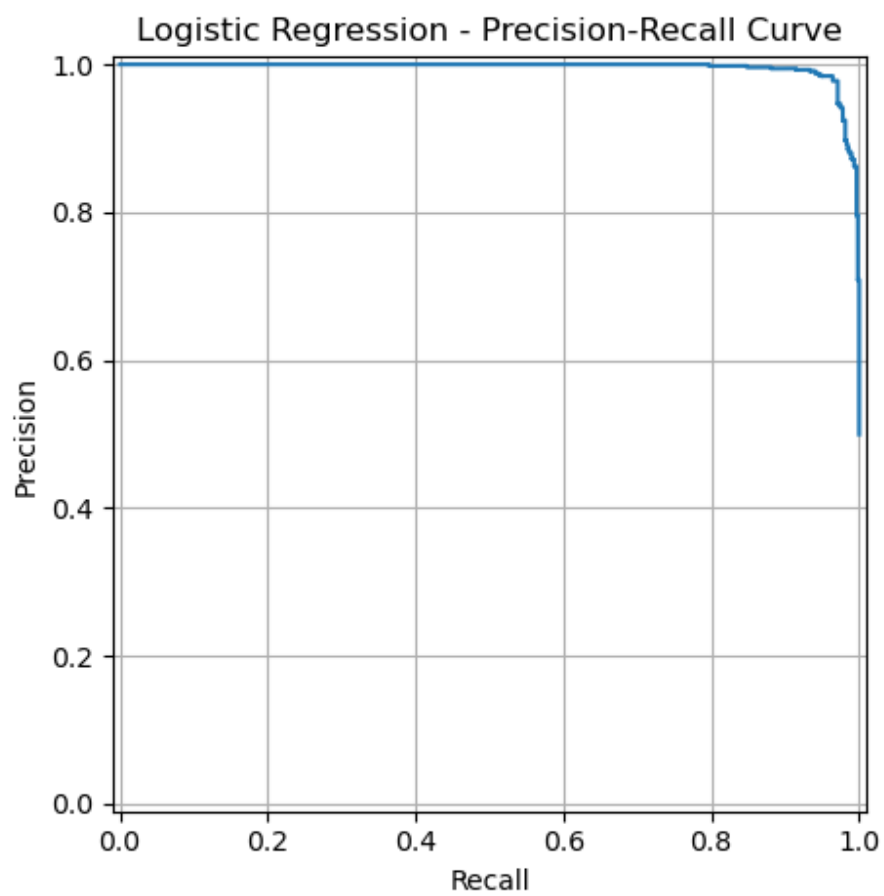


Figure A.5: Precision recall curve of XGBoost on the hold-out test set.

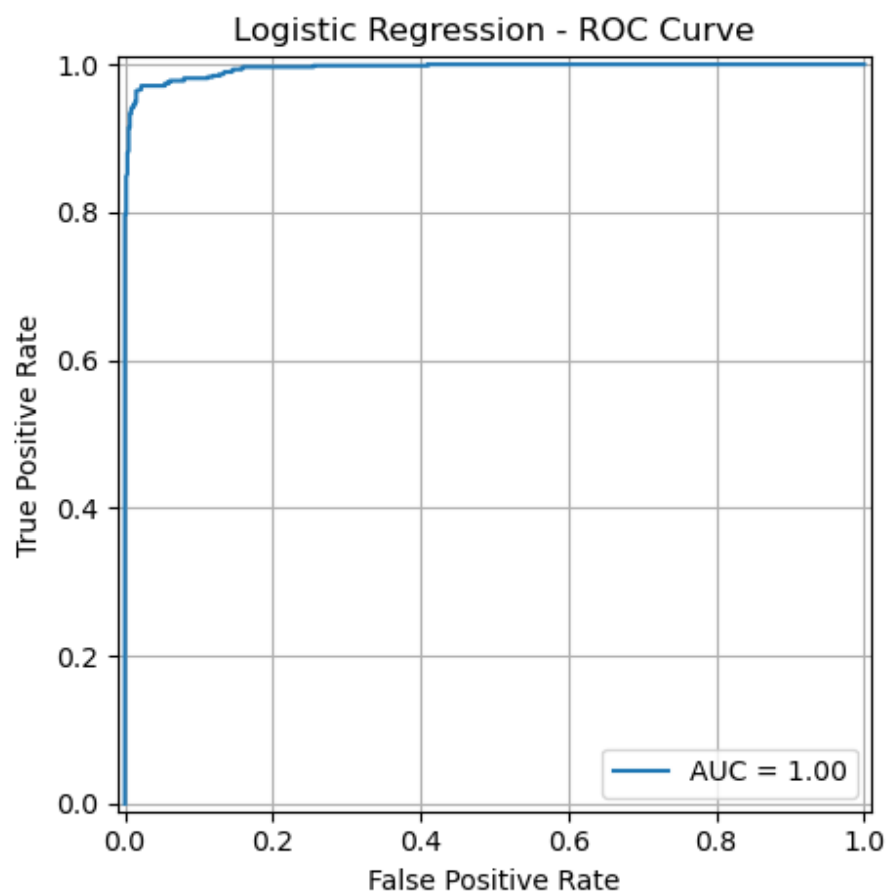


Figure A.6: Receiver operating characteristic curve of XGBoost on the hold-out test set.